Aprendizaje Federado

Extensiones de Machine Learning

Ginés Carrillo Ibáñez
Yago Ibarrola Lapeña
Aarón Palomar Peña

Universidad de Murcia
Facultad de Informática

Curso 2025/2026

Contenidos

1 Sesión 1 **2**
1.1 FL centralizado 2
1.2 FL descentralizado 2

2 Sesión 2 **4**
2.1 Model poisoning 4
2.2 Label flipping 4
2.3 Comparación y análisis 5

3 Sesión 3 **6**
3.1 Trimmed Mean 6
3.2 Krum 7
3.3 Comparación y análisis 7

1 Sesión 1

Para esta sesión, junto con la instalación de Nébula, se han realizado los siguientes cuatro experimentos sobre el entrenamiento de una CNN en el dataset CIFAR-10 de clasificación de imágenes. Se han realizado 10 rondas de entrenamiento en una única época, se han desplegado 4 nodos y las actualizaciones se han calculado usando FedAvg. Los experimentos realizados han sido:

- Entrenamiento centralizado con datos homogéneos (IID) y heterogéneos (non-IID) entre los nodos.
- Entrenamiento descentralizado con datos homogéneos (IID) y heterogéneos (non-IID) entre los nodos.

1.1 FL centralizado

Comparamos el entrenamiento IID y non-IID entre los nodos. En el caso non-IID, se utiliza una distribución de dirichlet con $\alpha = 0.1$. Se puede ver la comparación de ambos casos en la Figura 1:

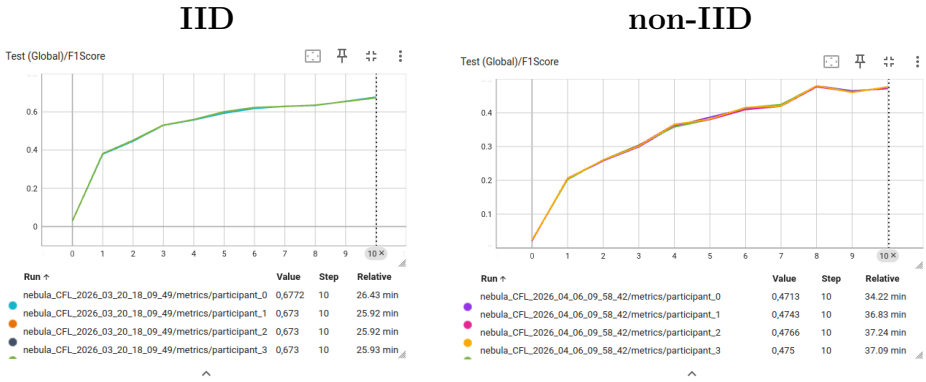

Figure 1: Comparación del entrenamiento centralizado con datos IID y non-IID.

Vemos que en el caso de datos homogéneos, el modelo converge más rápidamente y alcanza una mayor precisión, llegando en la ronda 10 a un F1-score de aproximadamente 0.65, mientras que en non-IID solo alcanza un F1-score de 0.47. Esto tiene sentido, ya que en el caso non-IID, cada nodo tiene una distribución de datos diferente, lo que dificulta la generalización del modelo global. Igualmente, en la ronda 3 IID ya alcanza un F1-score de 0.52, mientras que non-IID solo llega a 0.3.

1.2 FL descentralizado

En este caso, se ha realizado el mismo experimento pero con un entrenamiento descentralizado. Se muestran los resultados en la Figura 2:

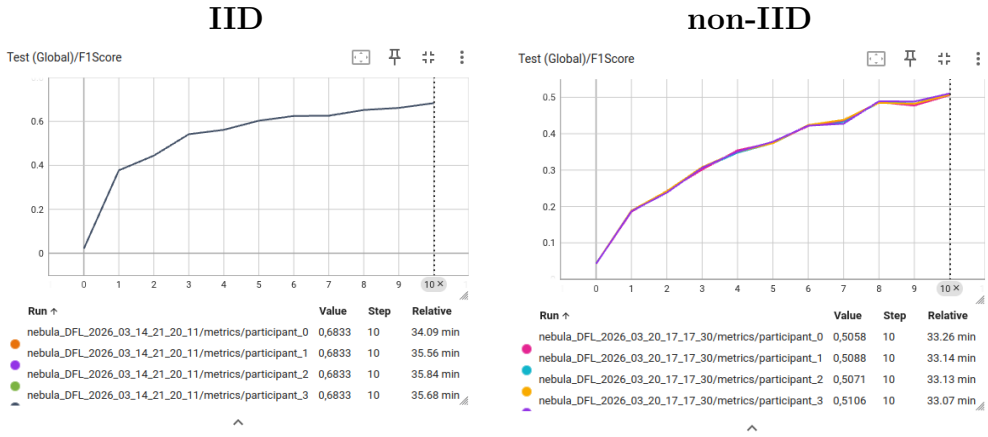

Figure 2: Comparación del entrenamiento descentralizado con datos IID y non-IID.

- Ronda 3: IID alcanza un F1-score de 0.5, mientras que non-IID solo llega a 0.3.
- Ronda 10: IID alcanza un F1-score de aproximadamente 0.67, mientras que non-IID solo alcanza un F1-score de 0.5.

En general, aunque CFL y DFL muestran resultados similares en términos de rendimiento (F1-score), presentan diferencias importantes a nivel de arquitectura. CFL destaca por su simplicidad y menor sobrecarga de red y recursos, pero introduce un punto único de fallo y posibles cuellos de botella en federaciones grandes. Por otro lado, DFL ofrece mayor resiliencia frente a fallos y ataques, a costa de una mayor complejidad en la gestión de modelos, sincronización para la agregación y comunicaciones entre nodos.

Podemos ver las matrices de confusión para ambos casos en la Figura 3:

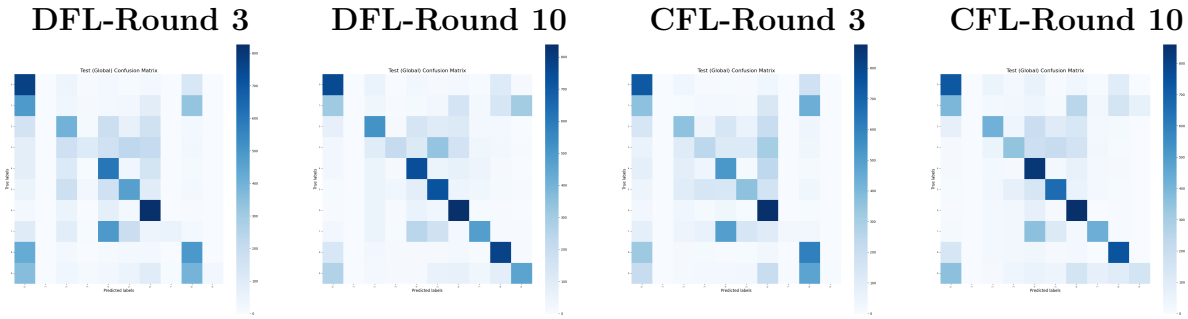

Figure 3: Matrices de confusión para el entrenamiento non-IID en DFL y CFL en las rondas 3 y 10.

Vemos que ambas rondas 3 son muy parecidas, donde todavía la diagonal no está muy marcado, con errores principalmente en las clases 0, 4 y 8. En la ronda 10 sí que se aprecian menos errores, con una diagonal más marcada.

2 Sesión 2

Para esta sesión, se ha utilizado de nuevo la configuracion de entrenamiento de DFL non-IID de la sesión anterior. Se han realizado con ella los siguientes experimentos:

- Ataque de *model poisoning* con 10% de ruido gaussiano.
- Ataque de *label flipping* con 80% de muestras envenenadas.

Para cada uno de los ataques, compararemos con un 20% y un 50% de clientes maliciosos.

2.1 Model poisoning

En este ataque, los clientes maliciosos añaden ruido gaussiano a sus modelos locales antes de enviarlos para la agregación. Este ruido degrada la calidad de las actualizaciones, afectando al proceso de convergencia.

Los resultados obtenidos se muestran en la Figura 4:

20% nodos maliciosos **50% nodos maliciosos**

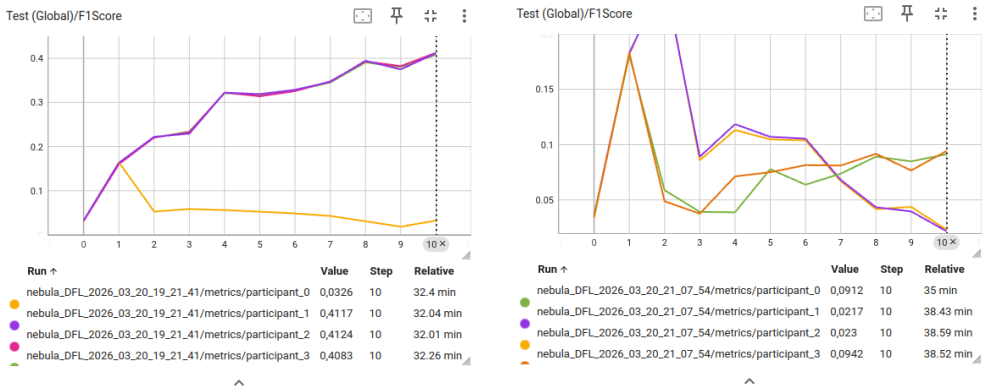

Figure 4: Impacto del ataque de *model poisoning* con diferentes porcentajes de nodos maliciosos.

Se observa que el impacto del ataque aumenta significativamente con el número de nodos maliciosos, provocando una notable inestabilidad:

- Con un 20% de nodos maliciosos, la mayoría de los participantes logra converger hacia un F1-score aproximado de 0.41. Sin embargo, el nodo atacante queda rezagado con un valor de apenas 0.03 porque realiza el test después de introducir el ruido al modelo recibido por el agregador.
- Con un 50% de nodos maliciosos, la convergencia se ve gravemente afectada. La gráfica muestra fuertes oscilaciones y una degradación final, desplomando el F1-score a valores casi nulos de entre 0.02 y 0.09 en el último paso.

2.2 Label flipping

En este caso, los nodos maliciosos alteran las etiquetas de un 80% de sus muestras de entrenamiento de manera aleatoria. Este ataque introduce información incorrecta directamente en el proceso de aprendizaje.

Los resultados se muestran en la Figura 5:

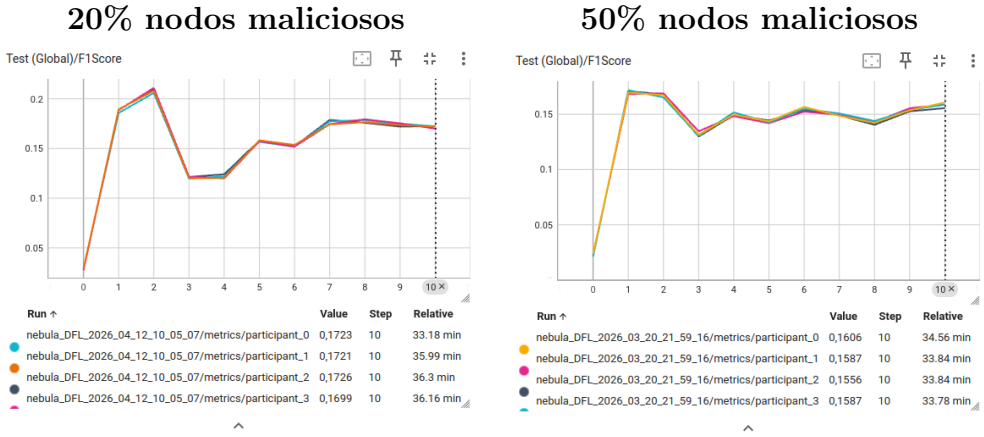

Figure 5: Impacto del ataque de *label flipping* con diferentes porcentajes de nodos maliciosos.

En este escenario, el impacto reduce el rendimiento pero mantiene un entrenamiento más estable:

- Con un 20% de nodos maliciosos, el modelo presenta una convergencia similar para todos los clientes, pero el rendimiento se desploma, alcanzando un F1-score final en torno a 0.17.
- Con un 50% de nodos maliciosos, el modelo logra converger estancándose de manera uniforme en un F1-score de 0.15.

2.3 Comparación y análisis

Comparando ambos ataques y configuraciones con los datos obtenidos, se extraen las siguientes conclusiones:

- Contrario a lo que se podría intuir en un inicio, el ataque de *model poisoning* de 10% genera un escenario más inestable con dos nodos maliciosos que el de *label flipping* al 80%. Mientras el primero reduce la convergencia (F1-score ≤ 0.09), el segundo logra engañar al modelo llevándolo a una convergencia falsa pero ligeramente superior (F1-score 0.16).
- A medida que aumenta el porcentaje de clientes maliciosos (20% $\rightarrow$ 50%), la métrica final cae en ambos ataques (drásticamente en el primero): de 0.41 a 0.09 en *model poisoning*, y de 0.17 a 0.16 en *label flipping*.
- En la sesión anterior, el entorno DFL ideal (IID) alcanzaba un F1-score de 0.67, mientras que la simple introducción de heterogeneidad en los datos (DFL non-IID base) lo reducía a 0.50. Al introducir condiciones de ataque sobre este último escenario, la degradación se amplifica: en la condición de ataque más leve (20% *label flipping*), el F1-score ya cae por debajo del caso base a 0.48. En las condiciones de ataque más severas (50% *model poisoning*), el modelo se desploma hasta un 0.02. Esto evidencia que los entornos non-IID son frágiles ante actualizaciones maliciosas, ya que a la agregación le resulta mucho más difícil distinguir entre una variación legítima (datos sesgados) y un ataque intencionado.

Matriz de atacante

3 Sesión 3

Para esta sesión, se ha utilizado de nuevo la configuración de entrenamiento DFL non-IID con la presencia de un nodo malicioso realizando *model poisoning*, tal y como se definió en la sesión anterior. Sobre este escenario, se han evaluado dos mecanismos de agregación robusta con el objetivo de mitigar el impacto de actualizaciones adversarias:

- Agregación mediante *Trimmed Mean*.
- Agregación mediante *Krum*.

3.1 Trimmed Mean

El algoritmo *Trimmed Mean* consiste en eliminar un porcentaje de los valores más extremos (potencialmente maliciosos) antes de realizar la agregación de los modelos, reduciendo así la influencia de outliers.

Los resultados se muestran en la Figura 6:

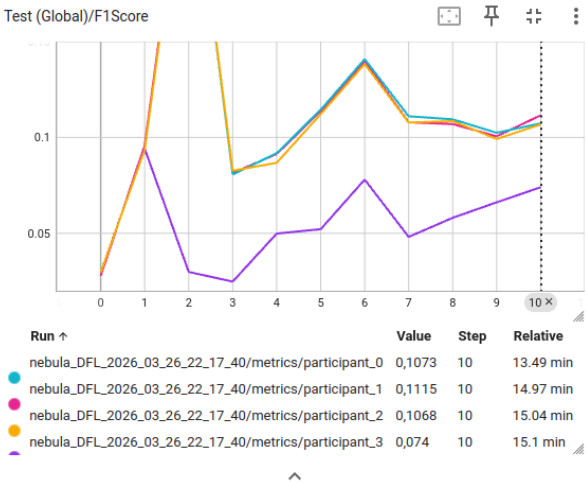

Figure 6: Resultados del entrenamiento con agregación *Trimmed Mean*.

Se observa que este método consigue mitigar parcialmente el efecto del nodo malicioso:

- El modelo converge de manera relativamente estable, evitando las oscilaciones bruscas observadas en *model poisoning* sin defensa.
- El F1-score final se sitúa en torno a 0.11–0.12, lo que supone una mejora significativa frente al caso atacado sin mecanismos de defensa.
- Sin embargo, el rendimiento sigue siendo inferior al escenario sin ataques, indicando que el filtrado de valores extremos no elimina completamente el impacto adversario.

Punto clave: TrimmedMean usa beta=0 por defecto en trimmedmean.py:15, y no vi que backend lo configure. Con beta=0, no recorta extremos y termina en media simple por componente en trimmedmean.py:37. Además, ese TrimmedMean no pondera por tamaño de dataset (solo promedia tensores), mientras FedAvg sí pondera por muestras en fedavg.py:23 y fedavg.py:33. Krum aquí selecciona un único modelo (el de menor suma de distancias) en krum.py:44 y krum.py:49; no pondera por muestras y puede tirar mucha información útil en non-IID.

3.2 Krum

El algoritmo *Krum* selecciona la actualización más representativa entre los nodos, basándose en la distancia entre modelos, con el objetivo de descartar aquellos que se alejan significativamente (potencialmente maliciosos).

Los resultados se presentan en la Figura 7:

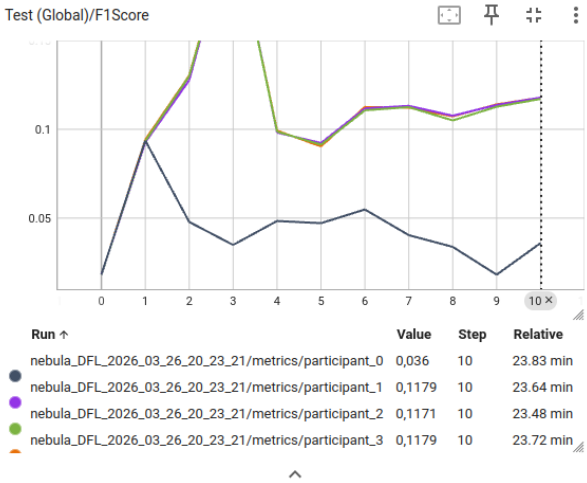

Figure 7: Resultados del entrenamiento con agregación *Krum*.

En este caso, el comportamiento es diferente:

- El entrenamiento es más estable que en el caso sin defensa, pero presenta mayor variabilidad que *Trimmed Mean*.
- El F1-score final se sitúa aproximadamente entre 0.07 y 0.11 dependiendo del nodo, mostrando un rendimiento ligeramente inferior en promedio.
- Esto sugiere que, aunque Krum es capaz de descartar actualizaciones maliciosas, puede verse afectado por la heterogeneidad non-IID, seleccionando modelos que no representan adecuadamente la distribución global.

3.3 Comparación y análisis

A partir de los resultados obtenidos, se pueden extraer las siguientes conclusiones:

- *Trimmed Mean* demuestra una mayor robustez frente a ataques de *model poisoning*, ya que consigue estabilizar la convergencia y alcanzar un F1-score superior en promedio.
- En escenarios con datos heterogéneos, *Krum* puede verse penalizado, ya que la variabilidad entre nodos puede confundirse con comportamiento malicioso, afectando negativamente a la selección de modelos.
- Ambos métodos mejoran claramente el rendimiento respecto al escenario atacado sin defensa (F1-score $\leq$ 0.09), pero no alcanzan los niveles del escenario sin ataques. Esto confirma que, aunque las técnicas robustas ayudan, no eliminan completamente el efecto adversarial.